

B

U

P

T

密码工程

第5讲、若干基础问题 (3) 素数与大数运算、信息论、时钟和存储秘密

授课教师：王晨宇

时间：2026年3月31日



目录

contents

B U P T



3.4

素数与大数运算

3.5

信息论

3.6

时钟

3.7

存储秘密

3.4

素数与大数运算



素数与大数运算



- ✓ 3.4.1 模运算
- ✓ 3.4.2 强素数
- ✓ 3.4.3 有限域
- ✓ 3.4.4 生成元
- ✓ 3.4.5 群



模运算 = 时钟算术

□ 我们可以将24h制的时间和12h制的时间来进行对比，两者的换算可以看作模12运算

上午7点 对应 上午7点 ($7 \bmod 12 = 7$)

下午15点 对应 下午3点 ($15 \bmod 12 = 3$)

晚上23点 对应 晚上11点 ($23 \bmod 12 = 11$)



同余

□ 如果 a 和 b 模 n 的余数相同，那么称 a 和 b 同余。

□ 例子：

□ $17 \equiv 2 \pmod{5}$ ，因为它们模 5 后都是 2

□ 写法为： $a \equiv b \pmod{n}$

□ 常见误区：

□ 同余 $\neq$ 相等

□ $17 \neq 2$ 但 $17 \equiv 2 \pmod{5}$

□ 同余只是余数相同，不是值相等。



完全剩余集

□ 对于数 n 来说，从 0 到 $n - 1$ 的所有整数，构成 n 的完全剩余集。

□ 例子：

□ 模 5 的完全剩余系是：{0, 1, 2, 3, 4}

$$18 \bmod 5 = 3、21 \bmod 5 = 1、25 \bmod 5 = 0$$

□ 结果永远落在 {0, 1, 2, 3, 4} 里

□ 为什么叫“完全”？

□ 因为这个集合包含了模 n 运算下所有可能的余数，一个都不少。



模运算加法

- 我们假定两个输入 a 和 b 都属于 p 的完全剩余集, 即 $\{0, 1, 2, \dots, p-1\}$ 。
- 加法非常简单
 - 1、把两个数相加。
 - 2、由于两个数的和不可能超过 $2p-1$, 因此只需减去 p , 即可得到正确结果。
- 例子:

$$a = 5, b = 3, p = 7$$

$$(5 + 3) \bmod 7 = 8 \bmod 7 = (8 - 7) = 1$$



模运算减法

□ 模 p 下的减法，就是做普通减法，如果结果是负数，加 p 即可。

□ 例子：

$$a = 3, b = 5, p = 7$$

$$(3 - 5) \bmod 7 = (-2) \bmod 7 = (-2 + 7) = 5$$

模减法的小提示

□ 负数余数 $\neq$ 数学错误

□ 它只是“没落在标准范围内”，加 p 归回区间。



模运算乘法

□什么是模乘法?

□就是 $(a \times b) \bmod p$ ，即先算出 $a \times b$ ，再模 p

□例子:

$$a = 3, b = 5, p = 7$$

$$(3 \times 5) \bmod 7 = (15) \bmod 7 = 1$$



模运算乘法

□为什么模乘更“麻烦”？

□相比加法和减法，模乘的结果增速更快，需要更频繁地模 p

□特别在 p 很小时，乘法结果很容易溢出

□实际技巧：

□某些场景下，可以提前“模化”输入：

$$[(a \bmod p) \times (b \bmod p)] \bmod p$$

□这样可减少运算中的大数，优化效率



模运算结合律

□ 模乘法中「先模后乘」和「后乘再模」效果一样

$$(a \times b) \bmod p = [(a \bmod p) \times (b \bmod p)] \bmod p$$

□ 例子：

□ $a = 27, b = 19, p = 5$

□ 直接算： $27 \times 19 = 513, 513 \bmod 5 = 3$

□ 分步算： $27 \bmod 5 = 2, 19 \bmod 5 = 4, 2 \times 4 = 8, 8 \bmod 5 = 3$

□ 提前模 p 能保证乘法结果更小，防止大数计算超限



模运算分配律

□ 模运算下，乘法对加法满足分配律

$$[a \times (b + c)] \bmod p = [(a \times b) \bmod p + (a \times c) \bmod p] \bmod p$$

□ 例子：

□ $a = 3, b = 5, c = 8, p = 7$

□ 左边： $5 + 8 = 13$ ， $3 \times 13 = 39$ ， $39 \bmod 7 = 4$

□ 右边：

□ $3 \times 5 = 15$ ， $15 \bmod 7 = 1$ ； $3 \times 8 = 24$ ， $24 \bmod 7 = 3$

□ $1 + 3 = 4$ ， $4 \bmod 7 = 4$



什么是强素数？

- 强素数 (Strong Prime) = 特殊的 “更难分解” 的素数。
- 普通素数：只能被1和自己整除的正整数，如 3、5、7、11.....
- 强素数： 满足一系列特殊条件的素数，它们用于提高 $n = p \times q$ 的分解难度。

为什么需要强素数？

- 防止某些高效的分解算法对 n 快速分解。
- 增强RSA等加密算法的安全性。



强素数的特性

- 如果 n 是两个素数之积(可分解为 $n = p \times q$), 那么 p 和 q 采用强素数将更可取。
- 强素数不仅本身是素数, 还满足下列条件:
- $p - 1$ 和 $q - 1$ 的最大公因子应该较小。
- $p - 1$ 、 $q - 1$ 都应有大的素数因子, 分别记为 p' 和 q' 。
- $p' - 1$ 和 $q' - 1$ 都应有大的素数因子。
- $p + 1$ 、 $q + 1$ 都应有大的素数因子。
- $(p - 1)/2$ 和 $(q - 1)/2$ 都应该是素数。(注意, 如果这一条件满足, 那么前两个条件也满足)。

强素数的“结构”更复杂, 比普通素数更难被针对性算法分解。



强素数的争议

- 有些人认为长度更重要，只用大素数足矣。
- 因为对现有算法而言，不论是否是强素数，都不易计算。
- 但强素数可作为“额外保险”。

建议

- 若环境允许，仍建议选用强素数。
- 在高安全要求场景，使用强素数更稳妥。



什么是有限域？

- 数学家将素数 p 的完全剩余集称为 p 的有限域，即 $\{1, 2, \dots, p - 1\}$ 。
- 关于有限域中的计算，即可使用模运算的各种性质（加减法、乘法、结合律、分配律）。

有限域在密码学中的作用

- 有限域是现代密码学的核心数学结构。
- 广泛应用于分组密码（如AES）、椭圆曲线密码（ECC）和伽罗瓦域运算



椭圆曲线加密中的有限域

- 在椭圆曲线加密（ECC）中，所有的“点”和“计算”都发生在一个有限域上。
- 我们选取一个素数 $p = 17$ ，这就确定了一个有限域 $F_{17} = \{0, 1, 2, \dots, 16\}$
- ECC里，椭圆曲线的标准公式是：

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

- 我们取：

$$y^2 \equiv x^3 + 2x + 2 \pmod{17}$$



$$y^2 \equiv x^3 + 2x + 2 \pmod{17}$$

怎么用？

- 发送方和接收方都在这条曲线上选“点”作为公钥或私钥。
- 这些点是 F_{17} 这个有限域内的坐标点，比如 (5,1)、(6,3) 等。
- 通信双方通过相同的坐标点，即可计算出相同的值。

**由于椭圆曲线保密，且实际应用中选取的 p 会非常大
因此对于攻击者而言想要破解密钥就非常困难**



什么是生成元？

□生成元 (Generator) 是模运算中的一个特殊数字。

□如果对于素数 p ，有一个 g

□使得：

□对于 1 到 $p - 1$ 的每个数 b ，都存在某个 a

□满足 $g^a \equiv b \pmod{p}$ ，

□那么 g 就是模 p 的一个生成元。

把 g 看成“魔法钥匙”，通过不断幂运算并对 p 取模，可以“生成”1 到 $p - 1$ 的所有数。



生成元的例子

□ 取 $p = 11$, $g = 2$

□ 如右图所示

□ 通过对 g 的不断幂运算并对 p 取模，就可以得到1 到 $p - 1$ 的所有数（此处即为1~10）。

g p

$$\begin{aligned} 2^{10} &= 1024 \equiv 1 \pmod{11} \\ 2^1 &= 2 \equiv 2 \pmod{11} \\ 2^8 &= 256 \equiv 3 \pmod{11} \\ 2^2 &= 4 \equiv 4 \pmod{11} \\ 2^4 &= 16 \equiv 5 \pmod{11} \\ 2^9 &= 512 \equiv 6 \pmod{11} \\ 2^7 &= 128 \equiv 7 \pmod{11} \\ 2^3 &= 8 \equiv 8 \pmod{11} \\ 2^6 &= 64 \equiv 9 \pmod{11} \\ 2^5 &= 32 \equiv 10 \pmod{11} \end{aligned}$$



怎么判断一个数是不是生成元？

□ 要想判断 g 是否是 p 的生成元，我们需要对 $p - 1$ 进行操作。

□ 1、先分解 $p - 1$ 的素因子，记为 $q_1, q_2, \dots, q_n$

□ 2、对每一个素因子 $q_i (i = 1, 2, \dots, n)$ ，计算 $g^{\frac{p-1}{q_i}} \bmod p$

□ 3、如果任意一次计算的结果为1，则 g 并不是 p 的生成元，反之，则是。



素数与大数运算 3.4.4 生成元

B U P T

- 1、先分解 $p - 1$ 的素因子, 记为 $q_1, q_2, \dots, q_n$
- 2、对每一个素因子 $q_i (i = 1, 2, \dots, n)$, 计算 $g^{\frac{p-1}{q_i}} \bmod p$
- 3、如果任意一次计算的结果为1, 则 g 并不是 p 的生成元, 反之, 则是。

例子 ($p = 11$)

□ 对 $g = 2$, $p - 1 = 10 = 2 \times 5$

□ 验证 $g = 2$:

□ $2^5 \bmod 11 = 10 \neq 1$

□ $2^2 \bmod 11 = 4 \neq 1$

□ 因此, $g = 2$ 是 $p = 11$ 生成元

□ 对 $g = 3$, $p - 1 = 10 = 2 \times 5$

□ 验证 $g = 3$:

□ $3^2 \bmod 11 = 9 \neq 1$

□ $3^5 \bmod 11 = 243 \bmod 11 = 1$

□ 因此, $g = 3$ 不是 $p = 11$ 生成元



什么是群？

□ 群就是一组 “元素” + 一种 “操作”

□ “操作” 必须满足 4个基本规则：

□ 封闭性：任意两个元素相加（或者相乘）后，结果还在这个集合里。

□ 结合律：计算顺序不影响结果，比如 $(a + b) + c = a + (b + c)$

□ 单位元：存在一个特殊的 “0” 或 “1” 这样的元素，让你和它运算，结果不变，比如 $a + 0 = a$ 。

□ 逆元素：每个元素都能找到一个 “对手”，两者运算后能回到单位元，比如 $a + (-a) = 0$



群在密码学中的作用

□ 在密码学里，群常常是“模素数 p 的加法”或“模素数 p 的乘法”

□ 通俗来说：

□ 群就是一种“有秩序的运算规则”，它让我们在数学世界中可以“放心大胆地”做加减乘除。



Diffie-Hellman 密钥交换中的群

- Diffie-Hellman (DH) 是一种经典的公钥密码协议，双方可以在不直接传递密钥的情况下协商出一个共享密钥。
- 这个协议就是基于模 p 乘法群来实现的。



群的具体用法:

- 选择一个素数 $p = 23$ 。（此值可以公开）
- 选择素数 p 的生成元 $g = 5$ 。
- 即在 $Z_{23}^* = \{1, 2, 3, \dots, 22\}$ 这个群中， g 通过幂运算能够生成所有元素。

这个群的特点:

- 封闭性：任意两个数相乘后再 $\text{mod } 23$ ，还是落在 $\{1, 2, 3, \dots, 22\}$ 中。
- 逆元：每个数都有逆元（例如 $3 \times 8 \equiv 1(\text{mod } 23)$ ）。
- 单位元：1 是单位元。
- 结合律：正常乘法满足结合律。



素数与大数运算 3.4.5 群

B U P T

具体的协议流程:

Alice

素数 $p = 23$ (公开值)

p 的生成元 $g = 5$

Bob

选择 $a = 6$,

计算 $A = g^a \pmod{p} = 5^6 \pmod{23} = 8$

选择 $b = 15$,

计算 $B = g^b \pmod{p} = 5^{15} \pmod{23} = 19$

$A = 8$

$B = 19$

计算 $K = B^a \pmod{p} = 19^6 \pmod{23} = 2$

计算 $K = A^b \pmod{p} = 8^{15} \pmod{23} = 2$

□ 此时, Alice和Bob便协商出了共同的密钥 $K = 2$

3.5

信息论



- ✓ 3.5.1 信息熵
- ✓ 3.5.2 语言信息率
- ✓ 3.5.3 完全保密的密码系统
- ✓ 3.5.4 信息论在密码系统中的应用



什么是信息熵？

- 信息熵衡量信息中的“不确定性”或“混乱程度”。
- 不确定性越大，描述这条信息所需的比特数就越多。
- 例子：
 - 一周中的7天，需要3个比特来唯一表示。

000	= 星期日
001	= 星期一
010	= 星期二
011	= 星期三
100	= 星期四
101	= 星期五
110	= 星期六
111	是未用的



熵的大小

- 信息熵越大，代表这条信息越难预测，越难被压缩。
- 信息熵越小，代表这条信息越容易预测，容易被压缩。
- 例子：
 - 性别（男/女）只需要1个比特（0或1）来表示，熵=1位。
- 密码分析中，熵越大，密码越难被破解。



如何计算信息熵？

□ 通常，一条消息的熵是 $H(M) = \log_2 n$ ，其中 n 是消息所有可能的值。

□ 例子：

□ 一条表示性别消息的熵是 $H(M) = \log_2 2 = 1$ 位，

□ 一条表示一周天数的熵为 $H(M) = \log_2 7 \approx 2.8$ 位，稍微小于 3 位。



什么是语言信息率？

□ 语言信息率（rate of language）是衡量一种语言平均每个字符所携带的信息量。

□ 定义公式：

$$r = H(M) / N$$

□ $H(M)$ 是消息的熵， N 是消息长度。



- 标准英语的语言信息率通常在1.0~1.5位/字母之间。
- 信息率与消息长度有关：
 - 短文本的信息率更高。
 - 长文本信息率更低。
- 语言信息率用于衡量语言的压缩潜力和信息密度。



什么是绝对信息率？

- 假定每一个字符串是等可能的，对每一个字母而言可被编码的最大位数。
- 定义公式：

$$H(M) = \log_2 L$$

- L 是语言中不同字符的数量。
- 英语中 $L = 26$ ，则 $R = \log_2 26 \approx 4.7$ 位/字母。



冗余度

- 从前面可以看出，英语的语言信息率（1.0~1.5）远远低于绝对信息率(4.7)。
- 这一点是很正常的，因为自然语言具有高冗余度。
- 冗余度D被定义为：

$$D = R - r$$

- 即绝对信息率-语言信息率
- 已知英语的信息率是1.3，其冗余度则为 $4.7 - 1.3 = 3.4$ 位/字母。这意味着每个英语字母携带3.4位的冗余信息。



- Shannon 提出的密码学模型：攻击者目的是获取密钥 K 或明文 P 。
- 在几乎所有实际的密码分析中，分析者在开始着手前，总有一些有关 P 的统计信息。
- 例子：
 - 在一封发给 Bob 的消息中，它在开头很可能用 Dear Bob。
 - 这当然不是绝对，但 Dear Bob 比 e8T&g[,m 更有可能



什么是完全保密

□完全保密的定义：

- 从密文中无法得知关于明文的任何信息。

- 仅当可能的密钥数目至少与可能的消息数目一样多时，完全保密才是有可能的。

□完全保密的条件：

- 密钥空间 $\geq$ 消息空间，且密钥一次一用，且不复用。



冗余度导致的破解风险

□ 密文中泄露信息的根源：

- 明文的冗余信息。

- 明文冗余越大，密码分析者越容易缩小搜索空间。

- 比如英语中常用的 “th”、“ing” 等组合，攻击者通过这些规律可以缩小对明文的猜测范围。

□ 对策：

- 良好的加密系统应降低明文的冗余，如**先压缩、再加密**。



如何计算密码系统的熵？

□ 密码系统熵的计算：

$$H(K) = \log_2 K$$

□ K 为密钥空间的大小。

□ 例子：

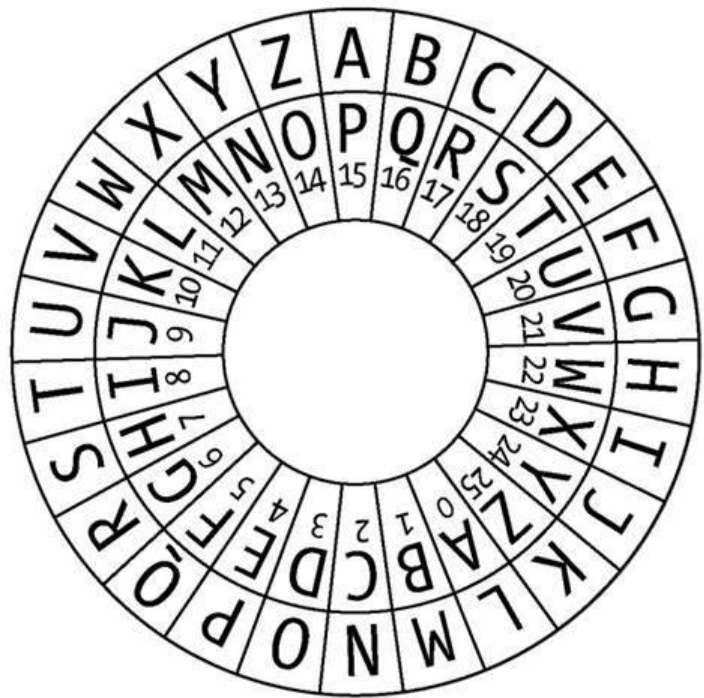
□ 64位密钥的密钥空间大小为 2^{64} ，因此熵为 $H(2^{64}) = \log_2(2^{64}) = 64$ 比特

□ 这意味着攻击者需要在 2^{64} 个密钥中穷举，难度非常高。



混乱

- 目的：隐藏明文与密文之间的直接对应关系，防止统计攻击。
- 典型方式：代替 (Substitution)
- 例子 (凯撒密码) :
 - 明文: "HELLO"
 - 偏移3位, 密文: "SPZZC"
 - 每个字母都被替换, 打乱明文与密文的直接关系。



凯撒密码



扩散

- 目的：将明文中的统计特征分散到整个密文中，降低可见性，即**打乱顺序**。
- 典型方式：置换（Permutation）
- 例子（列换位密码）：
 - 明文：“HELLO WORLD”
 - 重排为：“DLOW RHELLO”
- 明文字符被打乱，难以通过局部特征破解。



课堂思考

B U P T

1.在传输敏感数据时，应该先压缩后加密，还是先加密后压缩？为什么？

3.6

时钟



时钟



✓ 3.6.1 为什么需要时钟

✓ 3.6.4 时钟的安全隐患

✓ 3.6.2 时钟的功能

✓ 3.6.5 如何产生可靠的时钟

✓ 3.6.3 实时时钟

✓ 3.6.6 如何选择时间



时钟

3.6.1 为什么需要时钟

时间管理的基础

- 时钟，就是计算机内置的时间。
- 我们需要时钟来同步不同操作的执行，确保系统按预定的顺序正确运行。

在加密系统中的应用

- 密码管理不仅仅依赖于加密算法本身，还需要时间来控制密钥生成、更新和验证等操作。



1) 有效期管理

- 许多文件和证件（如票据、飞机票、证书等）都有有效期。
- 为了确保文件是否在有效期内，需要使用时钟来实时获取当前时间。

2) 生成唯一值

- 时钟能提供唯一的瞬时值，用于生成唯一标识符。
- 在安全信道中，时钟值用于确保每次使用的瞬时值都是唯一的，从而避免重复使用相同的时间戳。



唯一值的两种生成方式

- 直接使用时钟值作为唯一标识符。
- 使用时钟生成对应的时间码。
- 通过时钟生成的瞬时值确保系统的安全性，如用于PRNG（伪随机数生成）中的应用，保证每次生成的随机数都是唯一且安全的。



3) 保持单调性

- 时钟的一个重要特性是它一直向前发展，不能停止或倒退。
- 这一特性在加密协议中尤为重要，因为它确保了信息的时间戳不会跨越协议的时间范围，避免了数据的篡改。

应用场景：

- 在加密协议中，时钟的单调性用来防止攻击者将过时的信息放入当前的协议中。
- 只有符合当前时间条件的信息才能被接受。



4) 实现实时交易

实例：

□ 电子支付系统：

- 为了支持实时支付，银行需要进行实时金融交易，并记录这些交易的时间。
- 时钟用于确保交易按照正确的顺序执行，防止交易时间发生错乱。

重要性：

- 如果时钟出错，可能会导致交易顺序错乱，甚至出现交易的错误记录。
- 例如，如果支付交易的时钟被设定为过去的时间，这会影响系统的正确性，导致新的交易无法完成。



什么是实时时钟？

- 实时时钟（Real-Time Clock, RTC）是计算机系统中一个负责跟踪当前时间的硬件组件。
- 它通过电池供电，即使计算机断电，依然可以持续计时。

常见用途：

- 操作系统启动：
 - 在计算机开机时，帮助操作系统确定当前日期和时间。
- 设备同步：
 - 用于多个设备之间的时间同步，保证它们在相同时间范围内执行任务。



为什么安全系统不能使用RTC?

1) 时钟错误的风险

- 在普通计算机或设备中，时钟错误通常影响不大，可能导致某些操作的时间戳错误。
- 在安全系统中，时钟错误可能严重影响系统的安全性，尤其是如果攻击者利用错误的时间戳篡改数据或篡改事件的顺序。



为什么安全系统不能使用RTC?

2) 安全系统对时钟的高要求

□ 高精度要求:

□ 安全系统需要极高精度的时钟，任何时间偏差都可能导致身份验证失败，或者误判数据的合法性。

□ 防止篡改:

□ RTC在某些情况下容易受到物理攻击或系统漏洞影响，攻击者可能通过修改时钟来绕过某些安全检查，导致系统漏洞。



1) 时钟停止攻击

- 每个设计者都会本能的认为时间是不会静止的。他们设计的系统也都假定时间在正常的运行。
- 但是如果时钟停止，很多系统会以不可预测的方式来运行。

例子：金融交易中的时钟停止攻击

- 系统使用时钟来标记每笔交易的时间，并以此为基础来处理账户的余额和交易。
- 攻击者使系统的时钟停止，一些交易可能会被重复计算，或者某些交易没有正确处理。



2) 时钟回置

- 攻击者可以通过将时钟设置为过去的某一时刻，来绕过某些访问控制。
- 这种攻击方式允许攻击者重新访问本不再可用的敏感数据，甚至是已经过期的会话数据。

例子：优惠券的时效性

- 假设某电商平台提供了一个优惠券，优惠券在3.31之前有效，可以享受一定折扣。
- **攻击者在4.1日将时钟回拨到3.31之前**，系统会错误地认为优惠券仍然有效，并允许攻击者使用已过期的优惠券。



3) 时钟前置

- 把时钟向前设置会让计算机认为自己生活在未来，这会引起拒绝服务攻击。
- 如果把时钟设成四年以后，将会有非常多的信用卡因为已到期而被拒绝。

例子：提前收税

- 在缴税的场景中，如果**将时钟的时间调到30年之后**，系统会开始对所有客户收缴30年未付的账，这种情况通常不会导致直接的经济损失。
- 但是倘若纳税人开通了自动扣费的功能，则会扣走纳税人账户的钱。



时钟

3.6.5 如何产生可靠的时钟

1) 基于计数器的方案

- 每次开机时，计算机会启动一个“计数器”，从零开始数。
- 它通过计数来记录从开机到现在的时间。
- 然而，该方式仅能对系统启动之后的时间变化进行跟踪，并不能保证与外部标准时间的一致性。



2) 跟踪最后关机的时间

- 每次关机时，系统会将最后的时间进行记录。
- 当再次开机时，与先前记录的时间进行比对。
- 这种方式用于防止系统的时间不会往前跳跃太多，至少不会早于上一次关机的时间。
- 但只使用这种方式并不能保证启动时获取到正确的时间。



3) 询问网络服务器

- 通过向网络服务器查询时间，获取标准的时间源，确保时钟与全球标准时间同步。

4) 时间同步协议

- 使用时间同步协议（如NTP或SNTP协议）与外部时间源（如GPS时间或原子钟）进行同步。
- 时间同步协议能够提供高精度的时间同步，广泛应用于需要精确时间的系统中，如PKI系统和金融交易系统。



时钟 3.6.6 如何选择时间

B U P T

不建议使用本地时间

- 不建议使用本地时间，因为本地时间会因季节变化而调整（如夏令时、冬令时的切换）。
- 这会带来许多不稳定因素，特别是在全球范围内运行的系统中。

UTC时间

- UTC时间是全球统一的时间标准，适用于所有计算机系统，因此能避免因为各地时间调整带来的问题。



UTC时间的“跳跃秒”问题

- 为了让UTC与地球自转同步，每隔几年会插入1秒（跳跃秒）。
- 大多数计算机系统标准时间为60秒，跳跃秒的加入会导致系统中有重复的1秒。
- 跳跃秒看似微小，但会破坏时间的单调性和唯一性。

如何解决

- 对于大多数应用，保持时钟的单调性和一致性比精确同步更重要。
- 只要能保证时钟永远不会往回跳一个跳跃秒，跳跃秒对大部分系统不影响。

3.7

存储秘密



存储秘密



✓ 3.7.1 秘密存储的要求

✓ 3.7.5 秘密共享

✓ 3.7.2 磁盘与便携式存储

✓ 3.7.6 Shamir秘密共享算法

✓ 3.7.3 加盐和扩展

✓ 3.7.7 秘密共享的意义与挑战

✓ 3.7.4 丢失的风险



存储秘密

3.7.1 秘密存储的要求

B U P T

保持隐私性

- 存储的秘密应保持其隐私性，不应泄露或被未授权访问。

减少丢失的风险要求

- 存储的秘密应该能够安全地恢复，避免丢失。

最小化访问权限要求

- 只有必要的用户和应用程序可以访问秘密，减少数据暴露的可能性。



计算机硬盘

□ **传统方式：** 密钥存储在PC或其他设备的磁盘上。

风险

- PC不安全，任何使用该PC的人都有可能获取密钥。
- 多设备使用，容易导致密钥复制分散，增加泄露风险。



U盘



硬件钱包

便携式存储

- 使用随身设备（手机、U盘、硬件钱包等）存储密钥。
- 减少密钥在多台设备中的分散，提升安全性。



什么是加盐？

□ 加盐 = 给密码加上一点“随机调料”。

例子：

□ 用户A的密码： $\text{Password}_A = \text{Hash}(\text{“123456”} + \text{盐A})$

□ 用户B的密码： $\text{Password}_B = \text{Hash}(\text{“123456”} + \text{盐B})$

□ $\text{Password}_A \neq \text{Password}_B$ ，在系统中存储的则是 Password_A 和 Password_B

加盐的作用就是为每个密码加上不同的“随机字符串”，让相同的密码最终的加密结果也不同，从而有效防止批量破解。



问题：

□ 如果黑客暴力破解，反复尝试一百万个密码，速度会非常快。

解决方案：扩展（Stretch）

□ 扩展 = 故意让加密计算更慢

□ 将“密码+盐”做多次加密（比如反复哈希上百万次）。



扩展的例子：

- 正常加密 = “密码+盐” ➡ 1次
- 哈希扩展加密 = “密码+盐” ➡ 哈希1次 ➡ 再哈希 ➡ 再哈希 …… 连续哈希上百万次！
- 黑客如果想测试一个密码，却又不知道具体的扩展次数，那么每进行一次扩展计算，都要测试一下当前的密码是否正确。



加盐 + 扩展 = 双保险

- 加盐：让每个用户的密码加密结果不同，防止批量破解。
- 扩展：让每次尝试一个密码都变得“费力”，拖慢攻击速度。

从数学角度看：

- 如果扩展设置 $r = 2^{20}$ （大约100万次哈希）
- 黑客每尝试1个密码，就需要执行 2^{20} 次哈希。
- 现代系统通常结合“加盐”和“扩展”，来大幅提升密码保护的安全性！



为什么普通口令不够安全？

- 加盐和扩展仅仅是拖慢攻击者破解密码的速度，口令仍有失效风险。
- 未来计算机变得更强，黑客可以用更快的速度破解今天的加密口令。

使用有前向安全性（**Forward Secrecy**）的密钥协商协议

- 即使攻击者在未来获取了密钥或口令，也无法解密过去的通信内容。
- 每次通信使用“**临时密钥**”，即使私钥泄露，也无法还原历史数据。



易用性 vs 安全性

□ **难点：** 密钥存储系统既要方便使用，又要高度可靠，难以两全。

□ **解决思路：**

□ 一个“易用”的备份，方便日常使用。

□ 一个“更可靠”的存储，确保意外时能找回。

□ **常见做法**

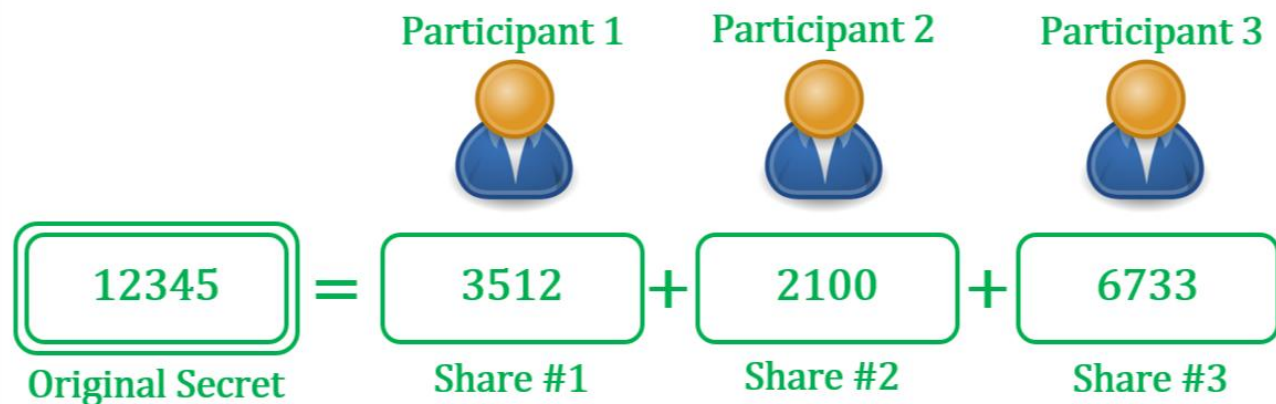
□ **易用副本：** 保存在常用设备、应用中，快速访问。

□ **可靠副本：** 存放在更安全的地方（如纸质存档、银行保管箱等）。



什么是“秘密共享”？

- 秘密共享（Secret Sharing）= 把一个秘密分成多份，每人一份。
- 单独一个人拿不到秘密，必须“凑齐”几个人才能复原。



最简单的秘密共享



算法概述

□ **选择秘密：** 假设秘密是 S ，只有当至少 k 个参与者合作时才能够解开秘密。

□ **构造多项式：** 构造 $k - 1$ 次多项式：

$$f(x) = S + a_1x + a_2x^2 + \cdots + a_{k-1}x^{k-1}$$

□ **分配秘密：** 计算多项式在不同 x 值上的结果，生成份额 $D_i = (x_i, f(x_i))$ 。

□ **恢复秘密：** 任意 t 个份额的集合通过拉格朗日插值公式恢复秘密 $S = f(0)$ 。



1、选择秘密

□ 假设秘密是 S ，只有当至少 k 个参与者合作时才能够解开秘密。

□ 我们假设秘密值 $S = 45$ ， $k = 3$ 。



2、构造多项式

□构造 $k - 1$ 次多项式：

$$f(x) = S + a_1x + a_2x^2 + \cdots + a_{k-1}x^{k-1}$$

□由于 $S = 45$ ， $k = 3$ ，我们构造2次多项式。

□对于 a_i ，可以随机选择系数 $a_1 = 19$ ， $a_2 = 13$ 。

□因此，构造的多项式为：

$$f(x) = 45 + 19x + 13x^2$$



3、分配子秘密

- 计算多项式在不同 x 值上的结果，生成份额 $D_i = (x_i, f(x_i))$ 。
- 假设能够共享秘密的人数为 5，那么就需要计算 5 份子秘密进行分配。
- 先任意取 5 个不等的 x 值： $x = \{1, 2, 3, 4, 5\}$ 。
- 子秘密分别为：

$$D_1 = (1, f(1)) = (1, 77)、D_2 = (2, f(2)) = (2, 135)、$$

$$D_3 = (3, f(3)) = (3, 219)、D_4 = (4, f(4)) = (4, 329)$$

$$D_5 = (5, f(5)) = (5, 465)$$



4、恢复秘密 (1)

□任意 t 个份额的集合通过拉格朗日插值公式恢复秘密 $S = f(0)$ 。

□假设子秘密的格式为： $D_i = (x_i, y_i)$ ，拉格朗日插值公式即为：

$$f(x) = \sum_{i=0}^n y_i l_i(x) \text{ , 其中, } l_i(x) = \frac{(x-x_1)(x-x_2)\dots(x-x_n)}{(x_i-x_1)(x_i-x_2)\dots(x_i-x_n)}$$

□例如，我们选取 $D_1 = (1,77)$ 、 $D_2 = (2,135)$ 、 $D_3 = (3,219)$ 来恢复秘密

$$\square \text{ 则 } l_1(x) = \frac{(x-2)(x-3)}{(1-2)(1-3)} = \frac{x^2-5x+6}{2} \text{ 、 } l_2(x) = \frac{(x-1)(x-3)}{(2-1)(2-3)} = \frac{x^2-4x+3}{-1} \text{ 、}$$

$$\square l_3(x) = \frac{(x-1)(x-2)}{(3-1)(3-2)} = \frac{x^2-3x+2}{2}$$



4、恢复秘密 (2)

□ 将 $l_1(x) = \frac{x^2-5x+6}{2}$ 、 $l_2(x) = \frac{x^2-4x+3}{-1}$ 、 $l_3(x) = \frac{x^2-3x+2}{2}$ 代入

$$f(x) = \sum_{i=0}^n y_i l_i(x)$$

$$\begin{aligned}\text{可得: } f(0) &= y_1 l_1(0) + y_2 l_2(0) + y_3 l_3(0) \\ &= 77 * 3 + 135 * (-3) + 219 * 1 \\ &= 45\end{aligned}$$

□ 即求出秘密值 $S = f(0) = 45$



为什么要用秘密共享？

- 防止单点泄露： 避免某1个人掌握全部密钥。
- 提升安全性： 只有多人合作，才能取回密钥或敏感数据。

秘密共享的挑战

- 对大多数企业，更倾向于使用“物理隔离”或“多重备份”。
- 秘密共享非常安全，但操作复杂、成本较高，适合关键密钥或少数高价值场景。



**2.你认为在现实生活中，哪些场景适合采用“秘密共享”？
哪些场景更适合用“多重备份”？**

谢谢大家
